Smaller Gate Counts for Polynomial-Basis Binary-Field Multipliers:

AES-field multiplication in 112 gates at depth 6 and 107 gates at depth 13, and explicit circuits for $\mathbb{F}_{2^4}, \mathbb{F}_{2^5}, \mathbb{F}_{2^{12}}, \mathbb{F}_{2^{13}}$

Swapnil Jain

Preprint, October 2026

Abstract

We report multiplier circuits over $\mathbb{F}_2$ with fewer two-input gates than the best circuits we could find in the literature, with different strength of comparison for different targets. Two targets improve the gate count of a published circuit (at greater depth for $\mathbb{F}_{2^7}$): $\mathbb{F}_{2^8}$ in the AES polynomial basis and $\mathbb{F}_{2^7}$ for x^7+x^3+1 (`0x89`). Four fields ($\mathbb{F}_{2^4}$, $\mathbb{F}_{2^5}$, $\mathbb{F}_{2^{12}}$, $\mathbb{F}_{2^{13}}$) beat only a constructed baseline and general published formula counts, not a published circuit, and $M(128)$ is an explicit circuit for a bound that can already be derived from published results. In the model where AND and XOR gates cost one each, we multiply in $\mathbb{F}_{2^8}$ (AES polynomial $x^8+x^4+x^3+x+1$) with 112 gates at depth 6, against 117 gates at depth 6 by Peralta et al. (the "CMT" circuits, 2010). Trading depth for size we also give 110 gates at depth 7, 108 at depth 12 and 107 (35 AND, 72 XOR) at depth 13; the 107 is more than twice as deep as the 117, so the like-for-like gain is 117 → 112. The same CMT page lists a 106-gate, depth-6 circuit for $\mathbb{F}_{2^8}$ in a tower basis that is not given on the page; it computes the product in a different basis, so using it in the AES polynomial basis needs basis conversions that cost extra gates, and it is not directly comparable. For $\mathbb{F}_{2^7}$ with x^7+x^3+1 we give 83 gates against 85, at depth 6 or 7 against 5. For binary polynomial multiplication we give an explicit 11464-gate circuit for 128×128 bits, a bound derivable from published results, against the table value 11486. We also give circuits for $\mathbb{F}_{2^4}$ (30), $\mathbb{F}_{2^5}$ (44) and the two Classic McEliece fields $\mathbb{F}_{2^{12}}$ (210) and $\mathbb{F}_{2^{13}}$ (283); each is below the published binary polynomial multiplier count $M(m)$ plus the cost of naive reduction (31, 49, 229, 303; a simple construction, not a published circuit, with depths not compared), and we found no published circuit file for these fields. General published formulas (Sunar and Koç for trinomials) give 31, 49 and 287 for $\mathbb{F}_{2^4}$, $\mathbb{F}_{2^5}$, $\mathbb{F}_{2^{12}}$, above ours; later low-complexity constructions may be smaller and were not evaluated. Every field circuit is checked exhaustively over all input pairs, symbolically, and against must-fail controls; $M(128)$, whose input space is too large to enumerate, gets only the last two checks. The AES-field circuits at depths 6, 7, 12, 13, 18 and 19, every circuit of Table 3, all three $\mathbb{F}_{2^7}$ circuits, and the $\mathbb{F}_{2^4}$, $\mathbb{F}_{2^5}$ and $\mathbb{F}_{2^6}$ circuits also have Lean 4 kernel proofs; the Lean statement is a decided Boolean check of a generated gate list (Section 2.2). The hashes of 13 of the 22 released circuits are committed in a weak public timestamp (an editable GitHub gist dated 2026-10-11) that shows existence on the day of this preprint, not earlier. The circuits come from three ingredients: a family of recursive product sets whose output map is solved as a linear system, linear layers built with Paar and Boyar–Peralta-type heuristics (newer heuristics were not applied), and depth rebalancing. We also report what did not work.

Keywords: binary fields, multiplication circuits, gate count, AES, Karatsuba, formal verification.

1 Introduction

A multiplier for $\mathbb{F}_{2^m}$ in a polynomial basis is a bilinear map of $2m$ input bits to m output bits. Circuit size, counted in two-input AND and XOR gates, is a standard cost for software bitslicing, masked and multi-party implementations, and logic synthesis starting points. At small sizes the records come from two sources. For plain polynomial multiplication $M(n)$ over $\mathbb{F}_2[x]$, the tables of Bernstein [3] and of Find and Peralta [2] together hold the best values we found for $n \leq 100$. Above 100 the best published circuits we found are in Bernstein's table, which has circuits for every $n \leq 1000$ and holds $n = 128$ (11486 gates); Find and Peralta supply the base values (e.g. $M(64) = 3673$) from which a smaller $n = 128$ count can be derived. For field multiplication the CMT page [1] lists circuits for $\mathbb{F}_{2^6}$, $\mathbb{F}_{2^7}$ and $\mathbb{F}_{2^8}$ dating from 2010, and a tower-basis $\mathbb{F}_{2^8}$ circuit of 106 gates from 2017 whose basis is not given on the page.

Our target is the $\mathbb{F}_{2^8}$ multiplier in the AES polynomial basis, whose published record was 117 gates (48 AND, 69 XOR) at depth 6. That circuit applies one Karatsuba level over three 4×4 schoolbook blocks and finishes with a single 8×48 output layer, which does recombination and reduction together. We view the whole circuit as "linear map, products, linear map", search over the product set, and for each choice solve for the output map exactly.

Prior work we build on. The recursive splitting is Karatsuba's [4], in the refined form of Bernstein [3]. Find and Peralta [2] search symmetric bilinear formulas and optimise the linear layers separately. Subquadratic formulas are given by Cenk and Hasan [11] and by Fan and Hasan [10]; the Toeplitz matrix–vector form of the latter is the main earlier way to merge the reduction into the bilinear part. Linear layers are costed with the heuristics of Paar [5] and Boyar and Peralta [7, 8], including the randomised variant RNBP [9]. Wang's lower bounds [14] are for polynomial-product and matrix tensors, not field multiplication, and concern the AND count only; they say nothing about total gates. Bilinear rank of field multiplication itself is studied by Barbulescu et al. [23] and Cotardo and Zullo [24]. Shortest-linear-program heuristics newer than Paar and Boyar–Peralta exist [18, 19, 20, 21]; we did not apply them (Section 3.3).

Contributions.

- An $\mathbb{F}_{2^8}$ (AES polynomial) multiplier with 112 gates at the depth (6) of the 117-gate CMT circuit, and a gate–depth Pareto set of four points down to 107 gates at depth 13, each verified exhaustively, symbolically and in Lean (a decided Boolean statement, Section 2.2) (Table 1, Table 2).
- An $\mathbb{F}_{2^7}$ multiplier for $x^7 + x^3 + 1$ with 83 gates against 85, and an explicit $M(128)$ circuit with 11464 gates, a bound derivable from published results, against the table value 11486.
- Explicit circuits for $\mathbb{F}_{2^4}$, $\mathbb{F}_{2^5}$, $\mathbb{F}_{2^{12}}$ and $\mathbb{F}_{2^{13}}$ that are below the baseline of the published $M(m)$ circuit plus naive reduction and below the general published formula counts; we found no published circuit file for them (Section 1.2).
- A verification pipeline that ends in Lean 4 kernel proofs, plus a weak public timestamp of the hashes of 13 of the 22 released circuits.
- An account of the negative results, including cases where we do *not* beat the record.

The same machinery applies to the AES S-box; those results are reported separately and are not part of this paper.

What we do not claim. We make no optimality claim for any circuit. Several of our circuits are deeper than the circuit they improve on, which the tables show. Our literature check covers the CMT page, the Bernstein and Find–Peralta tables, and the sources in the bibliography; it is not a systematic survey.

1.1 Records at a glance

Verification level: **L** = exhaustive over all 2^{2m} input pairs + symbolic (exact algebraic normal form of every output) + must-fail controls + Lean 4 kernel proof; **V** = the same without Lean; **S** = symbolic + must-fail controls only (the input space is too large to enumerate). "Prev." is the best circuit we found in the literature in the same model and basis; for $M(128)$ it is Bernstein's table circuit, and the bound 11464 that can be derived from published results is named in the row but is not a published circuit. For the four fields without a published circuit it is a baseline: the published $M(m)$ of Bernstein and Find–Peralta (`sources/lit/RECORDS.md` in the pack: 25, 41, 207, 255) plus the XOR cost of naive reduction modulo the field polynomial (Section 1.2); the "published formula" column gives a general published formula count, not a circuit.

Target	Prev. (gates / depth)	Published formula	Ours (gates)	AND+XOR	Depth	Level	Verdict
$\mathbb{F}_{2^8}$, 0x11B	117 / 6 [1]	—	112	$36 + 76$	6	L	beats, same depth
		—	110	$36 + 74$	7	L	beats, deeper
		—	108	$34 + 74$	12	L	beats, deeper
		—	107	$35 + 72$	13	L	beats, deeper
		—	107	$35 + 72$	19	L	beats, deeper
		—	107	$36 + 71$	18	L	beats, deeper
$\mathbb{F}_{2^7}$, 0x89 (x^7+x^3+1)	85 / 5 [1]	—	83	$32 + 51$	7	L	beats, deeper
		—	83	$32 + 51$	6	L	beats, deeper
$\mathbb{F}_{2^7}$, 0x91 (x^7+x^4+1)	84 / 5 [1]	—	87	$32 + 55$	11	L	*does not beat*
$\mathbb{F}_{2^6}$, 0x49	57 / 5 [1]	—	57	$27 + 30$	5	L	re-derivation of CMT (same products)
$M(128)$ over $\mathbb{F}_2[x]$	11486 / 16 [3]; 11464 derivable	—	11464	$3888 + 7576$	16	S	explicit circuit, bound derivable (−22)
$\mathbb{F}_{2^4}$, 0x13	$25 + 6 = 31$ (baseline)	31 [15]	30	$10 + 20$	5	L	below naive baseline
$\mathbb{F}_{2^5}$, 0x25	$41 + 8 = 49$ (baseline)	49 [15]	44	$20 + 24$	6	L	below naive baseline
$\mathbb{F}_{2^{12}}$, 0x1009	$207 + 22 = 229$ (baseline)	287 [15]	210	$78 + 132$	17	V	below naive baseline
$\mathbb{F}_{2^{13}}$, 0x201b	$255 + 48 = 303$ (baseline)	not computed	283	$94 + 189$	29	V	below naive baseline

Table 1: Results against the best circuits or baselines we found. "Published formula" is the general gate count of the Mastrovito trinomial multiplier of Sunar and Koç [15], m^2 AND + $(m^2 - 1)$ XOR; for 0x201b (a pentanomial) the Reyhani-Masoleh–Hasan formula [16] was not computed. The $\mathbb{F}_{2^6}$ circuit uses the same 27 products as the CMT `slp_630` and is a re-derivation, not an independent result. The 107-gate circuit at depth 13 is a depth-rebalanced circuit derived from the depth-19 one; all three 107s have Lean proofs. The 107 at depth 13 is compared with a 117 at depth 6; the like-for-like comparison is the 112 row. The CMT 106-gate, depth-6 $\mathbb{F}_{2^8}$ circuit is for a tower basis not given on the page; using it in the AES polynomial basis needs basis conversions, whose cost we did not compute, so we do not compare with it. $\mathbb{F}_{2^{12}}$ and $\mathbb{F}_{2^{13}}$ use the Classic McEliece polynomials $x^{12} + x^3 + 1$ and $x^{13} + x^4 + x^3 + x + 1$.

1.2 Baseline for the four small fields

Published polynomial multipliers give an obvious baseline: take the best $M(m)$ circuit and reduce modulo the field polynomial. We computed the naive reduction cost with a short script (`tools/naive_reduction.py` in the pack) that folds each high coefficient c_k, $k =$

$2m-2, \ldots, m$, into the lower coefficients through the non-leading terms of the polynomial, one XOR per fold, with cascading folds counted, and checks the result against direct reduction. It gives 6, 8, 22 and 48 XORs for 0x13, 0x25, 0x1009 and 0x201b, so the baselines are $25+6 = 31$, $41+8 = 49$, $207+22 = 229$ and $255+48 = 303$ gates, against our 30, 44, 210 and 283. These baselines are not published circuits for the fields: the naive-reduction baseline is a simple construction from published ingredients (the reduction layer is not optimised), and depths are not compared. The comparison is kept as a size reference only. We found no published circuit file for these fields on the CMT page or in the Bernstein and Find–Peralta tables. General published formulas do give counts. Sunar and Koç [15] give m^2 ANDs and m^2-1 XORs, i.e. $2m^2-1$ gates, for the Mastrovito multiplier over any irreducible trinomial: 31 for 0x13 ($m = 4$), 49 for 0x25 ($m = 5$) and 287 for 0x1009 ($m = 12$). Reyhani-Masoleh and Hasan [16] give formulas covering pentanomials such as 0x201b; we could not extract the exact formula from the paper, so that count is not computed. Our 30, 44 and 210 are below these formulas and below $M(m)$ plus naive reduction. Later low-complexity trinomial and subquadratic constructions (e.g. Cho et al. [17]) may be smaller than the formulas above and were not evaluated, so a smaller published count may exist. Paar's 1994 thesis [6] and the Classic McEliece and McBits software contain multipliers for $\mathbb{F}_{2^4}$, $\mathbb{F}_{2^{12}}$ and $\mathbb{F}_{2^{13}}$ whose total gate counts we did not extract.

2 Gate model and verification

2.1 Gate model

A circuit is a straight-line program over two-input gates AND and XOR (XNOR is allowed by the checker but none of the circuits reported here uses it), each of cost 1. Inputs are the bits $a_0, \ldots, a_{m-1}$ and $b_0, \ldots, b_{m-1}$ of two field elements in a polynomial basis. We use no constants or negations, and fan-out is free. Depth is the length of the longest gate path, with every gate counting one. Totals are always AND + XOR with no extra weight on ANDs, but we list both counts. For $\mathbb{F}_{2^m}$ with irreducible P, the circuit must output the coefficients of $ab \bmod P$. For $M(n)$ the output is the $2n-1$ coefficients of the carry-free product.

2.2 Verification

Each circuit is checked by the same program (`verify.py`, in the artifact pack) with the three tests below; the Lean proof is a separate fourth check run with a different tool.

1. **Exhaustive.** All 2^{2m} input pairs (65,536 for $\mathbb{F}_{2^8}$) are evaluated bit-sliced, 2^{16} pairs at a time in Python integers. The reference outputs come from a separate bit-sliced formula (sum of $a_i b_j$ and the reduction matrix), which is itself spot-checked against a scalar carry-less multiplication followed by reduction.

2. **Symbolic.** Every signal is computed as its exact algebraic normal form over $\mathbb{F}_2$; each output must equal the reduced bilinear form $\sum_{i+j=k} a_i b_j$ exactly. This test alone covers $M(128)$, where 2^{256} pairs cannot be enumerated.

3. **Must-fail controls (checker-sanity tests).** A checker that accepts everything is useless, so we also apply mutations that must be rejected: a flipped gate type, a rewired operand, two swapped input labels and a check against a wrong polynomial (e.g. 0x11D for the AES circuits). These show that the checker can reject; they do not test the circuit

independently, since once the main check passes the wrong-polynomial and swapped-input controls fail by construction. All of them are rejected for every circuit in the pack (12 of 12 in the default setting of the released pack; earlier runs of `verify.py` on the 112-gate depth-6 circuit used 46, `sources/results/CANDIDATES.md` in the pack). For $M(128)$, which is not a field, there is no wrong-polynomial control; the twelfth control swaps two output bits instead.

4. **Lean 4 kernel proof.** For the circuits marked L, the theorem as written in each Lean file is `circuit_ok : allOkC = true`, where `allOkC` is a conjunction of decided Boolean chunks; chunk j checks that `evalPt` of the circuit equals `gmul` $p\,a\,b$ (the shift-and-add reference multiplication with reduction) for all $b < 2^m$ and a block of consecutive values of a. The layout is 64 chunks of 4 values of a for $\mathbb{F}_{2^8}$ (p = 0x11B), 64 chunks of 2 for $\mathbb{F}_{2^7}$, 64 chunks of 1 for $\mathbb{F}_{2^6}$, 32 chunks of 1 for $\mathbb{F}_{2^5}$ and 16 chunks of 1 for $\mathbb{F}_{2^4}$. Unfolding the definitions, this is equivalent to the statement that circuit(a, b) = `gmul` 0x11B $a\,b$ for all $a, b < 256$; we did not prove that equivalence as a separate theorem. Each chunk is proved with `decide +kernel`. The gate lists in the Lean files are generated from the `.slp` circuit files by `tools/gen_lean.py` (included in the pack). The `.lean` files are hashed in `lean/SHA256SUMS` but are not under the commitment root. The same file proves two controls *false*, both stated on the unchunked `allOk` rather than on `allOkC`: the first AND gate turned into an XOR, and the circuit against the wrong polynomial 0x11D. The wrong-polynomial control shows that the statement distinguishes the two polynomials, and the single mutant shows that a changed circuit is rejected; neither is independent evidence that the circuit is correct. For the main theorem and both controls `#print axioms` lists only `propext`, and neither `sorry` nor `native_decide` appears. We used core Lean v4.34.1 without Mathlib; the pack's `lean/` folder records the Lean version in `lean/lean_version.txt`. Where the run was timed (`lean/*.out`), each $\mathbb{F}_{2^8}$ proof took between 14m7s (the 108 at depth 19) and 17m45s (the 107 at depth 13): for example 17m41s for the 112 at depth 6, 14m26s for the 110 at depth 7, 17m3s for the 115-gate CMT-products circuit and 15m7s and 15m8s for the two 107s at depths 18 and 19, with about 3 GB of memory. The $\mathbb{F}_{2^7}$ proofs took 3m16s and 3m17s for x^7+x^3+1 (depths 7 and 6) and 2m40s for x^7+x^4+1, 24 seconds for $\mathbb{F}_{2^6}$, 4.4 s for $\mathbb{F}_{2^5}$ and 1.0 s for $\mathbb{F}_{2^4}$; every run has a logged time (`Aes109`: 16m08s). All runs were on one x86_64 Linux machine, and `lean/lean_version.txt` records the Lean version (the pack's `lean/SHA256SUMS` lists hashes of the Lean files and outputs). We chunked the statement because an unchunked 65,536-point `decide` hit a kernel timeout. Lean was run on the circuits marked L in Table 1 (the $\mathbb{F}_{2^8}$ circuits of 112, 110 and the 108 at depth 12, the three 107s at depths 13, 18 and 19, all three $\mathbb{F}_{2^7}$ circuits, the $\mathbb{F}_{2^6}$ circuit and the $\mathbb{F}_{2^4}$ and $\mathbb{F}_{2^5}$ circuits), and on every candidate of Table 3. It was not run on $\mathbb{F}_{2^{12}}$, $\mathbb{F}_{2^{13}}$ or $M(128)$. The $\mathbb{F}_{2^7}$, $\mathbb{F}_{2^6}$, $\mathbb{F}_{2^4}$ and $\mathbb{F}_{2^5}$ files use the same template with the respective field polynomials and the chunk layouts given above.

2.3 Commitment

We recorded SHA-256 hashes of the committed circuit files (listed in Table 6) in a file `SHA256SUMS`, in two rounds, both published in GitHub gists dated 2026-10-11. The first root, `f9cfddf4...`, is in the gist `3b2c5d8c...`; the first-round file is in the pack as `commitment/SHA256SUMS.v1` and hashes to that root. The second root, which supersedes it, is

`7139c329b0b76fe96c95e9012c4523c2ca5220a41bde366bc0dc0e800efaf078`,

in the gist `https://gist.github.com/Swapnil-jain/6750e3f460658bc51c9a5f376c93af72`; it added the 107-gate depth-13 and 108-gate depth-12 circuits to the first. A gist is editable by its owner and its date is GitHub's, so this is a weak timestamp: it indicates that these files existed on 2026-10-11, the same day as this preprint, and is not evidence of priority over earlier work. Only 13 of the 22 released circuits are under the root, and the seven S-box files that `SHA256SUMS` also lists are not in the pack, so only the 13 circuit hashes can be recomputed from it.

3 Method

3.1 The layered form

A circuit has three layers: a top layer of XORs computing linear forms $\alpha_k(a)$ and $\beta_k(b)$, a layer of K ANDs forming the products $p_k = \alpha_k(a)\beta_k(b)$, and a bottom layer of XORs mapping the K products to the m outputs. Once the product set is fixed, the bottom map $W \in \mathbb{F}_2^{m\times K}$ is the solution of a linear system: the a_ib_j monomials of every output are linear combinations of the products. We solve for W against the *reduced* target. Recombination and reduction modulo P then form a single matrix, with nothing appended afterwards. A merged output layer is not new in itself: the CMT 117 and the work of Fan and Hasan already merge recombination and reduction. What we do is solve for W exactly for every product set in the search and score every product set by total gates. The total cost is

$$K + L(\alpha) + L(\beta) + L(W),$$

where $L(\cdot)$ is the number of XORs needed for a linear layer. For a given product set a layered floor is $2K - m + d(\alpha) + d(\beta) + d(W)$; it is a counting heuristic that we use to rank product sets, and we do not prove it is a lower bound on built circuits. Here K counts the ANDs, $K - m$ is the minimum number of bottom XORs even with no sharing, $d(\alpha)$ and $d(\beta)$ are the numbers of distinct non-singleton top forms, and $d(W)$ (the number of distinct non-singleton columns of W) is used as an estimate of the *excess* of the bottom layer over $K - m$; the top-layer terms (one XOR per distinct non-singleton form, Appendix B) and the $K - m$ term are bounds. Appendix B spells this out. We recomputed the floor from the two 107-gate circuits with the anatomy script of our search (`sources/ideas/anatomy.py` in the pack): $2 \cdot 36 - 8 + 10 + 10 + 21 = 105$ for the 36-AND circuit and $2 \cdot 35 - 8 + 11 + 11 + 22 = 106$ for the 35-AND circuit, against 107 built.

3.2 Split families

Product sets are sampled from recursive splits of the coefficient index. Writing an index in binary, each index bit is assigned a *schoolbook* split (S, four products of the two halves) or a *Karatsuba* split (K, three products). For $m = 8$ the family $S \times K \times K$ gives $4 \cdot 3 \cdot 3 = 36$ products. Our 107-gate circuit with 36 ANDs has this form, with nine 2×2 schoolbook leaves on planes spanned by sums such as (a_i, a_{i+1}) and $(a_i + a_{i+2}, a_{i+1} + a_{i+3})$. In the 35-AND circuit one leaf is replaced by Karatsuba. The CMT 117-gate circuit has the factors the other way round ($K \times S \times S$, $3 \cdot 16 = 48$ products). The step from 117 to 107 is one more Karatsuba dimension together with the search over product sets scored by total gates. The building blocks are Karatsuba's splitting trick [4] and Bernstein's refined recombination [3]. The Find–Peralta recurrences [2] minimise the AND count first. We instead score every product set

by total gates, which favours more ANDs when they make the linear layers cheaper (the 117 itself uses 48 ANDs).

3.3 Linear layers

For a fixed product set the top forms and the bottom matrix are realised by shortest-linear-program heuristics: Paar's greedy common-pair elimination [5], Boyar–Peralta distance-based heuristics [7, 8], and the randomised variant RNBP [9]. We run them on the bottom matrix of every candidate and keep the best. We also build the bottom layer from a circuit for the *transposed* matrix (the *transposed-bottom builder*). By the transposition principle [22], for a linear map without zero rows or columns, the *optimal* fan-in-2 XOR counts of the map and of its transpose differ by exactly the difference between the numbers of rows and columns; the heuristic counts we obtain need not satisfy this. Run on the bottom layer of the CMT 117-gate circuit, the same heuristics save exactly one gate ($61 \to 60$ XORs, so 116 gates in total, Section 5); rebuilding both the top and the bottom layers on the CMT products gives 115 gates at depth 15 (Table 3). Layers alone therefore do not explain the gap to 107, which comes from the product sets. We did not apply the newer shortest-linear-program heuristics of Tan and Peyrin [18], Maximov [19], Xiang et al. [20] or Lin et al. [21] to our layers, so the linear layers of the 112, 107 and CMT 117 circuits may be further reducible.

3.4 Flip-graph walks scored by total gates

Flip graphs [12, 13] walk between tensor decompositions of the same rank. We adapted the idea to the product sets above and scored each visited decomposition by its estimated total gates (products plus the cost of the three linear layers) instead of by rank alone. A walk of about 9.4 million iterations reached the estimate 107 at $K = 35$, matching the circuit in hand. It also visited states at $K = 32$ to 35 with estimates of 108–109; none was below 107. Annealing over the orbit of the symmetries of the product set re-found 107 many times but never anything lower.

3.5 Window SAT

To look for local savings that the layered view cannot see, we cut a connected window of $w = 4, \ldots, 10$ gates out of a circuit and ask a SAT solver (Glucose via PySAT) for $w - 1$ gates computing the same outputs from the signals available before the window. Signal values are exact degree-2 algebraic normal forms (137 bits at $m = 8$), so the encoding is independent of the 2^{16} truth-table size. Equal-size replacements ("plateau moves") are accepted with probability 0.3 in half of the runs, and every accepted circuit is re-verified exhaustively and symbolically. The runs reported in Table 4 are on the circuits listed there (the smallest is the 109-gate circuit); the 107-gate circuits are not in the table. Section 5 gives the outcome.

3.6 Depth rebalancing

Our smallest circuits have depth 18–19. To fill in the size–depth trade-off we wrote `depth.py`, which rebalances XOR trees and resynthesises the top and bottom layers under a depth cap with a Paar-type heuristic. It runs for two rounds, with many randomised tries per cap; Table 2 shows the result. A depth-capped window-SAT pass found no improvement; it ran during depth rebalancing, and per-circuit statistics were not recorded.

4 Results

4.1 $\mathbb{F}_{2^8}$ with the AES polynomial

Gates	AND	XOR	Depth	SHA-256 (first 16 hex)	File	Level
112	36	76	6	af49795ac6920cdf	aes112_d6.slp	L
110	36	74	7	906f14db4a0cbff5	aes110_d7.slp	L
108	34	74	12	a23d03e1a6dabb68	aes108_d12.slp	L
107	35	72	13	cade935d4534dee0	aes107_d13.slp	L
Reference: CMT [1], 117 gates (48 AND, 69 XOR), depth 6.						

Table 2: Gate–depth Pareto points for 0x11B: the best gate count we found at each depth, kept only if it beats every shallower point. Best found under a depth cap: 112 at cap 6, 110 at caps 7 to 11, 108 at 12, 107 at 13.

At the depth of the published circuit (6) we save 5 gates (4.3%), 117 → 112. The 10 gates (8.5%) saved by the 107 compare a depth-13 circuit with a depth-6 one. Along the way the product-set search produced a ladder of verified, Lean-checked circuits at 114 (43 AND, depth 9), 113 (depth 15), 112 (depth 14), 110 (depth 20), 109 (33 AND, depth 18, Lean-checked), 108 (depth 19) and two 107-gate circuits (35 and 36 ANDs, depth 19 and 18); the depth-6, 7, 12 and 13 points of Table 2 come from rebalancing these. Of the two 107s, the 36-product circuit spends $10 + 10$ XORs on the top forms, 36 ANDs, and 51 XORs on the bottom. The 35-product circuit spends $11 + 11$ on the top and 50 on the bottom. The top layers have no slack: their size equals the number of distinct forms. The output map W is forced for each product set (kernel dimension 0), but the XOR count of the bottom layer comes from heuristics, so each 107 is the best we found for its product set, not a proven optimum; the layered floors are 105 and 106 (Section 3.1 and Appendix B). If the floor is a good guide, a gain below it has to change the products.

Candidate	Gates	AND	XOR	Depth
CMT products, our layers (aes_cmtprods_t115) (Lean-checked)	115	48	67	15
split family, solved W (aes_split) (Lean-checked)	114	43	71	9
aes_t113 (Lean-checked)	113	43	70	15
aes_t112 (Lean-checked)	112	43	69	14
aes_t_s1 (Lean-checked)	110	32	78	20
aes_t_s2 (Lean-checked)	109	33	76	18
aes_x_s3 (Lean-checked)	108	34	74	19
aes107_k35 (Lean-checked)	107	35	72	19
aes107_k36 (Lean-checked)	107	36	71	18
CMT, for reference [1]	117	48	69	6

Table 3: Verified 0x11B candidates before depth rebalancing (the 115-gate candidate is built from the CMT products with our top and bottom layers). All passed the exhaustive, symbolic, control and Lean checks. Our layers alone on the CMT products give 117 → 115, at depth 15. The AND count then shifts from 43–48 to 32–36: at depth 6 the 112 needs the new product set, and the further savings below 115 also come from changing it.

4.2 $\mathbb{F}_{2^7}$, $\mathbb{F}_{2^6}$

For $x^7 + x^3 + 1$ we obtain 83 gates (32 AND, 51 XOR) at depth 7 and at depth 6, against the CMT 85 at depth 5. Both circuits are Lean-checked. We did not find 83 at depth 5, and where depth matters the CMT circuit is still the better one. For $x^7 + x^4 + 1$ our best is 87, which does *not* beat the CMT 84 and is listed only for completeness. For $x^6 + x^3 + 1$ we match the CMT 57 without improving it. Our circuit uses the same 27 products as the CMT `slp_630` (three 3×3 blocks and the two sums) with the same AND/XOR split and depth, so it is a re-derivation of the CMT circuit, not an independent result. The count equals the Bernstein value $M(6) = 57$ (`sources/lit/RECORDS.md`), although the circuit also performs the reduction; we have no explanation beyond the search result.

4.3 Polynomial multiplication $M(128)$

Using the Find–Peralta base circuit $M(64) = 3673$ and the recurrence $M(2n) \leq 3M(n)+7n-3$ we build an explicit circuit with $3 \cdot 3673 + 7 \cdot 64 - 3 = 11464$ gates (3888 AND, 7576 XOR), depth 16, against the value 11486 in Bernstein's table. The number 11464 could already have been derived on paper from published ingredients (Find–Peralta's $M(64)$ and the recurrence); all we add is an explicit circuit file, and we do not call it a record. It is verified *symbolically* (with must-fail controls) but not exhaustively and not in Lean.

4.4 $\mathbb{F}_{2^4}$, $\mathbb{F}_{2^5}$, $\mathbb{F}_{2^{12}}$, $\mathbb{F}_{2^{13}}$

These come from the same pipeline (split-family sampling with solved output map, exact or Paar/BP layers, the transposed bottom, then linear-layer optimisation, window SAT and depth variants). We ran three sampling sweeps. The third found nothing new, and during polishing only $\mathbb{F}_{2^{12}}$ moved ($213 \to 210$). Window SAT produced no output on $\mathbb{F}_{2^{13}}$. We found no published circuit to compare against, so these numbers are upper bounds; the comparison we can make is with the $M(m)$-plus-reduction baseline and the general published formula counts of Section 1.2. The Classic McEliece fields are the reason we included them. Their depths (17 and 29) were not optimised.

5 Negative results and what did not work

- **GF(2^7), $x^7 + x^4 + 1$:** 87 gates against 84.
- **Large $M(n)$:** one-level Karatsuba plans on the Find–Peralta bases do *not* beat Bernstein for $n = 113$ (9703 vs 9534), 163 (17563 vs 16923), 233 (31176 vs 29354), 256 (35285 vs 34079) or 283 (42704 vs 38735). For these sizes Bernstein uses richer splits than we do.
- **Representation sweep (nothing found below, within the S/K split family over polynomial bases).** We rebuilt the split family in every polynomial basis of $\mathbb{F}_{2^8}$ (all 240, eight S/K schemes each), folding the basis change into the input forms and re-solving W. The AES basis is the only point with layered floor 105. The next best polynomial-basis point has floor 114, and nothing else is under 110. Among 300 random tower bases the best floor was 118. Flipping one leaf between S and K in every basis gave nothing under 105. The floor-105 family has a best built circuit of 107. This is

a statement about the sampled S/K split family over polynomial bases and the floor heuristic, not a proof that nothing lower exists.

- **Local product-set moves.** Re-choosing any single 2×2 leaf (up to 82 options) and exhaustive search of one leaf pair gave no circuit below 107. Among relabellings of the bit cube, only the 48 cube symmetries span the output space, and none of them does better. Adding any product from the form set gives kernel 0 and no saving, and there is no 2-for-1 product exchange within the form set.
- **Output-layer-first search (CEGAR) is not viable with our encodings at $\mathbb{F}_{2^8}$.** Here the output layer is chosen first and products and helper forms are searched for afterwards. We tried SAT and annealing for that search. Direct CDCL encodings did not finish within 40 seconds even on $\mathbb{F}_{2^4}$ with loose bounds, so only the annealing selection was used. It re-found our 30 and 44 for $\mathbb{F}_{2^4}$ and $\mathbb{F}_{2^5}$. At $\mathbb{F}_{2^8}$ the annealer could not re-find the 107 from its own pools (Hamming distance 6 to 15 of 512 after $2 \cdot 10^9$ steps), so target counts 104–106 gave no information.
- **CRT and twisted product sets.** Chinese-remainder product sets and "twisted" splits gave 114 at best in the runs we kept, no better than the first split-family circuit.
- **Window SAT** (Table 4). Windows of up to 10 gates over frontier signals were locally optimal almost everywhere. We tried 12 to 35 thousand windows per circuit, of which 93–98% were proved UNSAT at $w-1$ and 3–7% timed out at 5 s. No window improved the 109-gate circuit, nor any window on the 113-, 114- or 117-gate circuits. Table 4 lists no run on the 107-gate circuits; the depth-capped pass of the rebalancing step (Section 3.6) found nothing, with no per-circuit statistics recorded. An early calibration batch with a different solver found one $117 \to 116$ improvement on the CMT circuit.
- **Linear-layer heuristics alone** (Table 5). Paar, BP and RNBP on the output layer of the CMT 117 (8×48, 61 XORs) reach 60 at best (RNBP), giving a 116-gate circuit. On a 13-term polynomial multiplier they reproduce the Find–Peralta 255 but do not beat it. On $\mathbb{F}_{2^8}$ the heuristics alone do not reach the product-set candidates.
- **Monolithic exact SAT** for a full $\mathbb{F}_{2^8}$ multiplier (16 inputs, over a hundred gates) is out of reach. In a related earlier study of a 32-gate circuit on 16 inputs, even a bound known to be satisfiable was not solved in three hours.
- **Depth.** Every point below 112 gates is deeper than the CMT circuit. At depth 6 the best we found is 112, and at depth 5 we found nothing below 117 gates.

Circuit	Runs	Windows tried	UNSAT at $w-1$	Timeout (5 s)	Improved
Bernstein $n=5$ (41)	8	26,944	26,209	735	0
Bernstein $n=7$ (81)	8	12,494	11,609	885	0
$\mathbb{F}_{2^8}$ CMT (117)	8	13,829	12,931	898	0
$\mathbb{F}_{2^8}$ ours (114)	8	13,702	12,804	898	0
$\mathbb{F}_{2^8}$ ours (113)	8	16,363	15,570	793	0
$\mathbb{F}_{2^8}$ ours (109)	12	34,292	33,425	867	0

Table 4: Window SAT (windows of 4 to 10 gates, 5 s limit per window, 30 minutes per process). "UNSAT at $w-1$" counts windows proven not to shrink; about a tenth of those were followed by an equal-size plateau move.

Circuit	Bottom XOR now	Paar	BP	RNBP (best)	Total after
$\mathbb{F}_{2^8}$ CMT 117 (8×48)	61	61	61	**60**	116
$\mathbb{F}_{2^8}$ ours 114	57	57	57	57	114
$\mathbb{F}_{2^8}$ ours 113	56	57	57	57	113
Bernstein $n = 5$	16	16	16	16	41
Bernstein $n = 7$	35	35	35	35	81
Bernstein $n = 9$	59	59	59	59	132
Bernstein $n = 13$ (25×106)	120	119	119	119	255

Table 5: Linear-layer heuristics on the output layer of published and own circuits, 10 seeds $\times$ 300 restarts, product set fixed; every rebuilt circuit was re-verified. For $n = 13$ the result 255 equals the Find–Peralta value, so it is not new.

6 Artifacts

The repro pack `repro/` (published at `https://github.com/Swapnil-jain/gf2m-circuits` together with this paper) holds the circuits, the Lean files and outputs, the verifier, a README with exact commands, and `sources/` with the evidence files cited here (`lit/RECORDS.md`, `results/*.md`, `ideas/anatomy.py`) and the third-party circuit files read by `verify.py --baselines`; the latter are data files from the CMT page and from Bernstein's page, kept with their origin URLs and authors in `sources/README.md`. Circuit format: one gate per line, `Ti = X + Y` (XOR) or `Ti = X x Y` (AND), inputs `Ai`, `Bi`.

File	Circuit	SHA-256
`aes107_k35.slp`	0x11B, 107, depth 19	`ebae5b576046b74552c243214b3ecb6711a9ab785bfc3de207a162dadc11da09`
`aes107_k36.slp`	0x11B, 107, depth 18	`7bdcaf469f0c433d5357db9d742f93d0eec089e3d5e7a996672bb97e0af7f90f`
`aes110_d7.slp`	0x11B, 110, depth 7	`906f14db4a0cbff517b2f893e965266677ef741448746ae71181a31cfd0d82e8`
`aes112_d6.slp`	0x11B, 112, depth 6	`af49795ac6920cdf3a1a907fed8e4d049ffe10ae385c418187815010393178c5`
`final_f73.slp`	0x89, 83, depth 7	`519d1835d18b1a8fe843b669dc89aaff1f6afcbe588c095d5ee641fd7906ae7a`
`final_f73_d6.slp`	0x89, 83, depth 6	`e704a7468be62e08e20ac0aff34906ba87a3585f6d6369449902b3d01d05b24c`
`final_f4.slp`	0x13, 30	`9fa1f95b59dace527daaa87c13b6d7cb76c134f5f71491977739d8d4b98e2ddd`
`final_f5.slp`	0x25, 44	`4c171804c7c9ca61e509331ae537775930bf5a2783452b2bd2c95c862b3e62db`
`final_f12.slp`	0x1009, 210	`e983be789bfd7c0080d923461a5dc9076a0cda72d0b5e0d42b4dc858fa20446e`
`final_f13.slp`	0x201b, 283	`a4eca82092bbf761dd3a097e354881eb40a3d2bc4da6e5751b8ba01f445bb4e3`
`M128.slp`	$M(128)$, 11464	`aa5594ff03e83e4e99573527121 5f4b972ebf68353f6eb0fd3c2cbb72918a4df`
`aes108_d12.slp`	0x11B, 108, depth 12	`a23d03e1a6dabb68c1fe0a178fce073684ae6f4160c922636b4854dac2b1d06b`
`aes107_d13.slp`	0x11B, 107, depth 13	`cade935d4534dee0ffe55a7196bcd9ed8c5fdb4ac86c1b3b37346dc16035c6b8`
Released, not under the commitment root (hashes are not in `SHA256SUMS`):		
`aes114_d9.slp`	0x11B, 114, depth 9	`c05434a139f21117e40330acaaba5fb1e2af02bbcff5d79dc40eb91ce26d98a3`
`aes113_d15.slp`	0x11B, 113, depth 15	`cd980adf977e32ba4771d053a3dc2aa2f1b5f53c10d17200e7491029e3cb26ef`
`aes112_d14.slp`	0x11B, 112, depth 14	`448b70479ec530d0b91b4d0729d14b018399d8aae3af292408b206c11fa542fc`
`aes115_cmtprods_d15.slp`	0x11B, 115, depth 15 (CMT products, our layers)	`ec38207d1fa1153d58fea45ab5d34a2bbd908277fee48959f58d5834705c1c19`
`aes110_d20.slp`	0x11B, 110, depth 20	`3d84794d5e2f4e52365550c439503da5b7fbcb6756c8c641f93e1a321770707f`
`aes109_d18.slp`	0x11B, 109, depth 18 (Lean file `Aes109`)	`30e169d760379373da257cdff85fb93cd7ffe4ce5e05136773dddbe974a3197a`
`aes108_d19.slp`	0x11B, 108, depth 19	`f0c61c1a94174c69e2020f49b2efcbfd26102a537aee0f9f3ee3e0b1bc99a923`
`final_f7.slp`	0x91, 87, depth 11	`0f6d4f4d09522b7d48dbd862ab37a9f4bb28d87d3debb5514381d99736c88115`
`final_f6.slp`	0x49, 57, depth 5	`73e4145105972dd7288a6be66aeee41c2d796eda13290860603aefe174a3c8f6`

Table 6: Circuit files and hashes. The first block and the two circuits after the rule are under the commitment root; the last block was added to the pack after the commitment and is not covered by it. The 115-gate circuit built from the CMT products (Table 3) is released but not under the root. Lean files and their outputs (`Aes107k35`, `Aes107k36`, `Aes107d13`, `Aes108d12`, `Aes108d19`, `Aes109`, `Aes110d7`, `Aes110d20`, `Aes112d6`, `Aes112d14`, `Aes113d15`, `Aes114d9`, `Aes115d15`, `Gf7_f73`, `Gf7_f73_d6`, `Gf7_f74`, `Gf6`, `Gf4Rec`, `Gf5Rec`) are in `repro/lean/`.

7 Conclusion

We treated a field multiplier as "linear forms, products, one merged output map" and scored product sets by total gates. This takes the $\mathbb{F}_{2^8}$ AES-basis multiplier from 117 to 112 gates at the old record's depth (6), and to 107 gates at depth 13. The smallest circuits are deeper, and the same method does not beat the record for $x^7 + x^4 + 1$ or for large $M(n)$. The surviving gains lie in a family in which our searches found nothing lower: the AES basis is the unique point of its family with layered floor 105, and neither local moves nor the window-SAT runs of Table 4 (up to the 109-gate circuit) improved the circuits they were run on. We make no optimality claim. Two questions are open: whether circuits of a different kind (non-symmetric or non-bilinear products, or a different basis for the whole circuit) can go lower, and how the 107 relates to a lower bound. We know of no non-trivial lower bound on total gates. The methods also apply to the AES S-box, which is reported separately.

Acknowledgements. Computational search and drafting were assisted by AI tools; all released circuits were machine-verified.

A A complete circuit: $\mathbb{F}_{2^4}$ in 30 gates

The following circuit multiplies in $\mathbb{F}_2[x]/(x^4+x+1)$ and is the smallest example of the layered form, with eight top XORs (4 per operand), ten ANDs and twelve bottom XORs. We list it to show the format of the artifacts. It passes exhaustive (256 pairs) and symbolic verification, all controls, and a Lean kernel proof. Inputs are `A0..A3`, `B0..B3`; "+" is XOR and "x" is AND.

```
T1 = A0 + A1
T2 = A0 + A2
T3 = A1 + A3
T4 = A2 + A3
T5 = B0 + B1
T6 = B0 + B2
T7 = B1 + B3
T8 = B2 + B3
T9 = A0 x B0
T10 = A2 x B2
T11 = T2 x T6
T12 = A1 x B1
T13 = A3 x B3
T14 = T3 x T7
T15 = T1 x T5
T16 = T1 x T8
T17 = T4 x T5
T18 = T4 x T8
T19 = T10 + T14
T20 = T12 + T9
T21 = T13 + T19
T22 = T21 + T20
T23 = T15 + T18
T24 = T23 + T14
T25 = T24 + T9
T26 = T11 + T20
T27 = T18 + T26
T28 = T16 + T17
T29 = T28 + T19
T30 = T29 + T26
C0 = T22
C1 = T25
C2 = T27
C3 = T30
```

B Why the output map can be solved, and what "floor" means

Fix forms α_k, β_k ($k = 1..K$). Each output c_j of the multiplier is a bilinear form in (a, b), hence a vector $t_j \in \mathbb{F}_2^{m \times m}$ of coefficients of $a_i b_j$. Each product p_k has the rank-one coefficient matrix $\alpha_k \beta_k^{\mathsf{T}}$. An output is realisable iff t_j lies in the $\mathbb{F}_2$-span of these K matrices, and the realising combination $W_{j,\cdot}$ is then unique exactly when the K matrices are linearly independent. In the circuits reported here the kernel dimension is 0, so W is forced and with it the function of the bottom layer. A circuit in this model therefore costs K plus the sizes of three linear layers, and what we search over is the set of product sets that span the targets.

For the floor: the two top layers need at least one XOR per non-singleton distinct form. The bottom map has K inputs and m outputs and must use every product, so it needs at least $K - m$ XORs even with no sharing. The floor is therefore $K + (K - m) + d(\alpha) + d(\beta) + d(W) = 2K - m + d(\alpha) + d(\beta) + d(W)$, where $d(\alpha), d(\beta)$ count distinct non-singleton top forms and $d(W)$ counts distinct non-singleton columns of W and serves as an estimate of the bottom layer's excess over $K - m$ (we have no proof that it is a bound). For the 36-AND circuit this is $64 + 10 + 10 + 21 = 105$ and for the 35-AND circuit $62 + 11 + 11 + 22 = 106$ (recomputed with `sources/ideas/anatomy.py` in the pack). A Karatsuba-type pair spends two extra top XORs to save a product (a symmetric pair costs 2 ANDs and 1 merge XOR; a Karatsuba replacement costs 1 AND and 2 new top XORs, with the squares' columns rewritten), which

is why the floor is roughly flat across $K = 34$ to 44. We only use the floor to rank product sets before building them; every reported count is of a built, verified circuit. The floor is a counting heuristic for ranking, unproven as a bound even on this layered form, and not a bound on total gates.

C Reproducing the numbers

The pack `repro/`, at `https://github.com/Swapnil-jain/gf2m-circuits` (made public together with this paper), contains a README with the exact commands. The verifier needs only Python 3 and takes about a second per circuit; the proofs need the Lean toolchain.

References

[1] J. Boyar, M. Dworkin, R. Peralta, M. Turan, C. Calik, L. Brandao (current members of the Circuit Minimization Team, CMT; past collaborators listed on the page include M. Bartock, R. Collazo, M. Find, M. Fischer, M. Cenk, C. Wood, A. Visconti, C. Schiavo, H. Gao, B. Strackbein, L. Bassham), *Circuit Minimization Work*, `https://www.cs.yale.edu/homes/peralta/CircuitStuff/CMT.html` (field-multiplier circuit files dated July 2010; page entries of September 2017).

[2] M. G. Find and R. Peralta, Better circuits for binary polynomial multiplication, *IEEE Transactions on Computers* 68(4), 2019, doi:10.1109/TC.2018.2874662.

[3] D. J. Bernstein, Batch binary Edwards, in *Advances in Cryptology – CRYPTO 2009*, LNCS 5677, Springer, 2009, pp. 317–336, doi:10.1007/978-3-642-03356-8_19; table of binary polynomial-multiplication circuits at `https://binary.cr.yp.to/m.html` (version 2009.05.31).

[4] A. Karatsuba and Yu. Ofman, Multiplication of multidigit numbers on automata, *Doklady Akademii Nauk SSSR* 145(2), 1962, pp. 293–294; English translation in *Soviet Physics Doklady* 7, 1963, pp. 595–596.

[5] C. Paar, Optimized arithmetic for Reed–Solomon encoders, in *Proc. IEEE International Symposium on Information Theory (ISIT)*, 1997, p. 250, doi:10.1109/ISIT.1997.613165.

[6] C. Paar, "Effiziente VLSI-Architekturen für bit-parallele Arithmetik in endlichen Körpern", Universität Essen (now Duisburg-Essen), 1994.

[7] J. Boyar and R. Peralta, A new combinational logic minimization technique with applications to cryptology, in *Experimental Algorithms (SEA 2010)*, LNCS 6049, Springer, 2010, pp. 178–189, doi:10.1007/978-3-642-13193-6_16.

[8] J. Boyar, P. Matthews, R. Peralta, Logic minimization techniques with applications to cryptology, *Journal of Cryptology* 26(2), 2013, pp. 280–312, doi:10.1007/s00145-012-9124-7.

[9] A. Visconti, C. V. Schiavo, R. Peralta, Improved upper bounds for the expected circuit complexity of dense systems of linear equations over GF(2), *Information Processing Letters* 137, 2018, pp. 1–5, doi:10.1016/j.ipl.2018.04.010.

[10] H. Fan and M. A. Hasan, A new approach to subquadratic space complexity parallel multipliers for extended binary fields, *IEEE Transactions on Computers* 56(2), 2007, pp. 224–233, doi:10.1109/TC.2007.19.

[11] M. Cenk and M. A. Hasan, Some new results on binary polynomial multiplication, *J. Cryptographic Engineering* 5(4), 2015, pp. 289–303, doi:10.1007/s13389-015-0101-6; ePrint 2015/094.

[12] M. Kauers and J. Moosbauer, Flip graphs for matrix multiplication, in *Proc. ISSAC 2023*, ACM, 2023, pp. 381–388, doi:10.1145/3597066.3597120; arXiv:2212.01175.

[13] S. Chen and M. Kauers, Flip graphs for polynomial multiplication, arXiv:2502.06264, 2025.

[14] C. Wang, Automated lower bounds for bilinear complexity over finite fields, arXiv:2603.07280, 2026 (background only; no result of this paper depends on it).

[15] B. Sunar and Ç. K. Koç, Mastrovito multiplier for all trinomials, *IEEE Transactions on Computers* 48(5), 1999, pp. 522–527, doi:10.1109/12.769434.

[16] A. Reyhani-Masoleh and M. A. Hasan, Low complexity bit parallel architectures for polynomial basis multiplication over GF(2^m), *IEEE Transactions on Computers* 53(8), 2004, pp. 945–959, doi:10.1109/TC.2004.47.

[17] Y. I. Cho, N. S. Chang, C. H. Kim, Y.-H. Park, S. Hong, New bit parallel multiplier with low space complexity for all irreducible trinomials over GF(2^n), *IEEE Transactions on VLSI Systems* 20(10), 2012, pp. 1903–1908, doi:10.1109/TVLSI.2011.2162594.

[18] Q. Q. Tan and T. Peyrin, Improved heuristics for short linear programs, *IACR Transactions on Cryptographic Hardware and Embedded Systems* 2020(1), pp. 203–230, doi:10.46586/tches.v2020.i1.203-230.

[19] A. Maximov, AES MixColumn with 92 XOR gates, IACR Cryptology ePrint Archive 2019/833.

[20] Z. Xiang, X. Zeng, D. Lin, Z. Bao, S. Zhang, Optimizing implementations of linear layers, *IACR Transactions on Symmetric Cryptology* 2020(2), pp. 120–145, doi:10.46586/tosc.v2020.i2.120-145.

[21] D. Lin, Z. Xiang, X. Zeng, S. Zhang, A framework to optimize implementations of matrices, in *CT-RSA 2021*, LNCS 12704, Springer, 2021, doi:10.1007/978-3-030-75539-3_25.

[22] M. Kaminski, D. G. Kirkpatrick, N. H. Bshouty, Addition requirements for matrix and transposed matrix products, *Journal of Algorithms* 9(3), 1988, pp. 354–364, doi:10.1016/0196-6774(88)90026-0.

[23] R. Barbulescu, J. Detrey, N. Estibals, P. Zimmermann, Finding optimal formulae for bilinear maps, in *Arithmetic of Finite Fields (WAIFI 2012)*, LNCS 7369, Springer, 2012, pp. 168–186, doi:10.1007/978-3-642-31662-3_12.

[24] G. Cotardo and F. Zullo, Symmetric tensor decompositions over finite fields, arXiv:2605.12295, 2026.